

gcc编译器

- 将两个C源文件编译成一个可执行文件: `gcc foo.c hello.c`
- 库文件链接: `gcc -I <头文件路径> foo.c hello.c -o hello`

Make 与 MakeFILE

- make的核心功能: **根据时间戳自动判断哪些部分需要重新编译, 每次只重新编译必要部分**
- MakeFile:人工指导文件编译链接成可执行文件

```
target: dependencies
    command 1
    command 2
    ...
    command n
```

- `$$` 代表当前目标文件名 (如 `out`、`case_add`) , 是 Makefile 最常用的自动变量;
- `$$` 代表所有依赖文件 (如 `case_all` 的依赖是 4 个样例文件) ; 典型样例:

```
.PHONY: all clean
all: case_add case_mul case_div
case_add: case_add.c
    gcc $$ -o $$
case_mul: case_mul.c
    gcc $$ -o $$
case_div: case_div.c
    gcc $$ -o $$
clean:
    rm -f case_add case_mul case_div
```

- MakeFile的嵌套: 内部文件夹的MakeFile提供接口
 - `make .C ./XXX <内部目标>`来指定进入哪个文件夹进行make
 -

linux命令

- 运行指令: `.\XXX` 必须要有目录, 不能为空, 否则只会在PATH中寻找
- 增加运行权限 (如.sh) : `chmod +x <文件名>`, `chmod +x *.sh`使用通配符

文件操作

- `../XXX`返回上一级寻找, 不需要拆开先`cd ..`

cat

cat 命令用于拼接文件内容, 是合并多个文件的基础操作。

grep 查找文件内容

```
grep [选项] PATTERN FILE
```

(PATTERN是匹配字符串，FILE是文件或目录的路径)

作用： 输出匹配PATTERN的文件和相关的行。

选项（常用）：

- -a 不忽略二进制数据进行搜索。
- -i 忽略大小写差异。
- -r 从目录中递归查找。
- -n 显示行号。

示例：file 文件含有 int 字符串所在的行数，并把 行号* 输出到 result 文件

```
grep -n int file | awk -F':' '{print $1}' > result
```

linux 文件权限

增加运行权限（如.sh）：`chmod +x <文件名>`, `chmod +x *.sh`

```
chmod [用户范围][操作符][权限符号] <文件名>
```

- 用户范围 包括：u (user/owner), g (group), o (others), a (all)。可省略，默认是a
- 操作符 包括：+ (添加), - (移除), = (赋值)。
- 权限符号 包括：r (读), w (写), x (执行)。要实现这个权限修改，最标准且高效的命令是：

权限字符串的拆解

- **r (read)**：读取权限，分值为 4
- **w (write)**：写入权限，分值为 2
- **x (execute)**：执行权限，分值为 1
- **- (none)**：无权限，分值为 0

以r--r-----为例子，首先拆成三组r--,r--,---- 于是

用户类别	权限字符	计算公式	八进制值
拥有者 (User)	r--	$\$4 + 0 + 0\$$	4
所属组 (Group)	r--	$\$4 + 0 + 0\$$	4
其他人 (Others)	---	$\$0 + 0 + 0\$$	0

或者我们可以采用符号表达

```
chmod u=r,g=r,o= stderr.txt
```

- **u=r**: 拥有者仅设为只读。
- **g=r**: 所属组仅设为只读。
- **o=**: 其他人权限全部清空 (等同于 **---**) 。

重定向与管道的妙用

- **<** 是输入重定向、**>** 是输出重定向
 - **>>** 指令追加输出到文件末尾，而不是覆盖原有内容。
 - **2> 2>>** **标准错误输出**
 - **常用！ 输出重定向如果没有目标文件，可以直接生成**
- **|** 是管道符，表示将前一个命令的输出作为后一个命令的输入。
-

sed 指令

```
sed [选项] '命令' <文件名>
```

选项 (常用) :

- **-n**: 显示经过sed处理的内容。否则显示输入文本的所有内容。
- **-i**: 真实修改，否则只显示。

命令 (常用) : **<行号>a<内容>**: 新增，在行号后新增一行相应内容。

- 行号可以是“数字”，在这一行之后新增，也可以是**起始行**，**终止行**，在其中的每一行后新增。
- 当不写行号时，在每一行之后新增。
- 使用**\$**表示最后一行。后面的命令同理。

<行号>c<内容>: 取代。用内容取代相应行的文本。 **<行号>i<内容>**: 插入。在当前行的上面插入一行文本。

<行号>d: 删除当前行的内容。 **<行号>p**: 输出选择的内容。通常与选项-n一起使用。

;使用分号一次多指令

s/str1/str2: 将所有每行第一个匹配的 str1 替换为 str2 **s/str1/str2/g**: 将每一个匹配的 str1 替换为 str2

示例: 多行输出连续行数 **sed -n '<起始行>,<终止行>p' <文件名>**

多行输出非连续 **sed -n '<行1>p;<行2>p....' <文件名>**

使用变量: 拼接字符串, 注意变量不要放到''中间

```
#命令bash modify.sh fibo.c char int,
#将 fibo.c 中所有的 char 字符串更改为 int 字符串。
sed -i 's/'$2'/'$3'/g' $1
```

awk 指令

表格化文档，以空格或者指定符号分割，输出某行某列 `awk options 'pattern {action}' file` 假设我们有一个叫 `student.txt` 的文件，内容如下：

张三 20岁 计算机系 默认切分（空格）：

- `awk '{print $1}' student.txt` $\rightarrow$ 输出 第一列 张三
- `awk '{print $3}' student.txt` $\rightarrow$ 输出 第三列 计算机系

如果想要指定切分

- 如，冒号前的 `awk -F":" '{print $1}' file`

Bash shell

- 命令行环境与解释器，类似于win中的批处理文件.bat `#!/bin/bash`选择bash解释器
- 输出： `echo`
- 变量： `a=1` 不允许有空格
 - 整数计算比较都需要`(( ))`
- 函数返回 `return $(( $1 + $2 ))`注意外面也需要`$`

典型案例：

- 自增 `$a=1` `$a=$(( $a+1 ))` 或 `let a=a+1`

GCC 编译器

`gcc -c <文件名> -o <XXX.o>` 生成目标文件机器码(.o) `gcc <文件名> -o <可执行程序名>` 生成可执行文件
`gcc -g <文件名> -o <可执行程序名>` 生成目标程序的 Debug 版本 `gcc <文件名>`默认生成一个out文件

- 多文件一起编译 `gcc <文件名1> <文件名2> ... -o <可执行程序名>`
- 头文件在其他文件夹 `gcc -I <头文件所在目录> <文件名> -o <可执行程序名>`
 - 注意是目录！

Vim

指令模式下：

- `$` 转到行尾
- `0` 转到行头
- `x` $\rightarrow$ 删当前光标所在的一个字符。
- `dd` $\rightarrow$ 删除当前行，并把删除的行存到剪贴板里
- `p` $\rightarrow$ 粘贴剪贴板
- `u` $\rightarrow$ undo `Ctrl+Z`
- `Ctrl+r` $\rightarrow$ redo

使用鼠标/Ctrl+q+上下 选中指定行后：

- `=` 自动缩进

在 Vim 的末行模式（按下:）输入：

- `%`: 代表当前文件名（如 `hello.c`）。

- %<: 代表去掉后缀的文件名（如 hello）。
- !: 告诉 Vim 这是一条外部 Shell 命令。

GDB 调试器

allows you to see what is going on 'inside' another program **while it executes**-- or what another program was doing at the moment it crashed.

第一步

- `gcc -g <file_name> -o <exe_name>` 生成debug版本的执行文件（否则的话是release版本） `gdb -tui <exc_file>` 启动TUI界面



基础调试

- 继续：对应 GDB 中的 `continue` 指令，一直执行到断点或者结束或者输入
- 步过：对应 GDB 中的 `next` 指令，执行当前所在的行，而**跳过当前行中的任何函数调用**
- 步入：对应 GDB 中的 `step` 指令
- 步出：对应 GDB 中的 `finish` 指令，从函数内部的执行跳出直接给出返回值。
- 重启：对应 GDB 中的 `start` 指令（在程序运行中使用）：跳转
- 停止：对应 GDB 中的 `kill` 指令

断点与参数

- 查看行号内容：`list [position]`: `list <line number>`、`list <file>:<line number>`、`list <function name>` 和 `list <file>:<function name>`
- 设置断点`break [position]`
- 设置临时断点 `tbreak` 暂停一次自动删除，`start`本质就是在main函数设置一个临时断点。
- 避免重复在这一断点处暂停：`disable breakpoints [num]...` num 需要查看断点编号`info breakpoints`
- 删除断点`delete breakpoints [num]...`
- 运行时加参数`start [arg]... run [arg]...`

条件调试

- 条件断点：`break/tbreak <position> if <condition>`
- 没有显式计数的条件断点：`tbreak <position> ignore <num> <times>` 忽略前K-1次的断点。

观察点-断点特殊类型

方便找到异常值产生位置！

- `watch <expression>` 设置观察点：当表达式取值发生变化，就会停下中断
- `awatch arr[n]` 设置访问观察点：对于一个N长度的数组，这可以观察是否发生数组越界的情况。
- `rwatch <expression>` 设置读观察点：

查看运行时数据

- `arr[<index>]@<len>` 表示显示数组中从<index> 处开始长度为<len>的范围内各元素的值。
- `disassemble -m <addr>` 或<函数名> 显示对应代码的汇编。
- `backtrace backtrace -n` 追溯显示栈帧，注意是负数！
- `frame <num>`查看第几个栈帧，追溯一些变量
- `print <expr> p <expr>`

修改运行程序

- `set <var> = <expr>`
- `return [expr]` 立即以指定的返回值返回
- `jump [pos]` 立即跳到（行或者函数）

Ctags

- 快速跳转到函数/类/结构体定义处。
- 在目录下运行`ctags -R *`
- 跳转`Ctrl + ]` 返回`Ctrl + O`

Tmux

终端窗口和进程分离，允许显示多个进程运行 `Ctrl+B` 松开后迅速 `SHIFT+5` 左右切屏 `tmux ls`查看会话名
`tmux a -t 会话名` 恢复到原来的会话

git

- `git restore`
 - `git restore <filename>` 未暂存时，回到上一次commit的状态
 - `git restore --staged <filename>` 暂存后，取消暂存
- `git checkout`
 - `git checkout lab<x>` 切换分支